Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

**SCHOOL OF COMPUTER SCIENCE ENGINEERING**
**WINTER SEMESTER 2025-2026**

**Assessment -4** **Slot: L5 + L6** **Class: VL2025260504771**

**Programme Name & Branch: B. Tech & CSE**

**Course Code & Title:** **BCSE309P – Cryptography and Network Security (LO)**

**Faculty Name: Dr. K. Murugan** **Maximum Marks: 10**
**Submitted by: Aryan Chopra** **Reg. no. :23BCI0144**

---

1. **(a) Develop a MD5 hash algorithm that finds the Message Authentication code.**

### 1 (a) MD5 Message Authentication Code

**Aim :**To generate a Message Authentication Code (MAC) for a given message using the MD5 hashing algorithm.

### Algorithm

1. Start the program.
2. Read the input message from the user.
3. Initialize MD5 context.
4. Update the context with the input message.
5. Finalize the MD5 computation to get the hash value.
6. Display the MD5 hash (MAC).
7. Stop.

## CODE

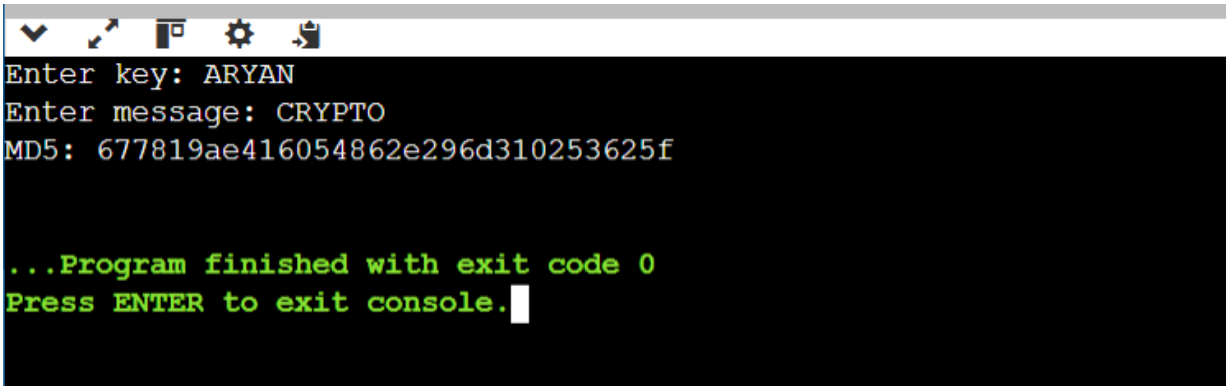
```
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <stdlib.h>
typedef struct {
uint32_t h[4];
```

```
uint64_t len;
uint8_t buf[64];
} MD5_CTX;
uint32_t leftrotate(uint32_t x, uint32_t c) {
   return (x << c) | (x >> (32 - c));
}

void md5_transform(MD5_CTX *ctx, uint8_t data[]) {
   uint32_t a, b, c, d, f, g, temp;
   uint32_t M[16];
   static const uint32_t s[] = {
      7,12,17,22, 7,12,17,22, 7,12,17,22, 7,12,17,22,
      5,9,14,20, 5,9,14,20, 5,9,14,20, 5,9,14,20,
      4,11,16,23, 4,11,16,23, 4,11,16,23, 4,11,16,23,
      6,10,15,21, 6,10,15,21, 6,10,15,21, 6,10,15,21
   };
   static const uint32_t K[] = {
      0xd76aa478,0xe8c7b756,0x242070db,0xc1bdceee,
      0xf57c0faf,0x4787c62a,0xa8304613,0xfd469501,
      0x698098d8,0x8b44f7af,0xffff5bb1,0x895cd7be,
      0x6b901122,0xfd987193,0xa679438e,0x49b40821,
      0xf61e2562,0xc040b340,0x265e5a51,0xe9b6c7aa,
      0xd62f105d,0x02441453,0xd8a1e681,0xe7d3fbc8,
      0x21e1cde6,0xc33707d6,0xf4d50d87,0x455a14ed,
      0xa9e3e905,0xfcefa3f8,0x676f02d9,0x8d2a4c8a,
      0xfffa3942,0x8771f681,0x6d9d6122,0xfde5380c,
      0xa4beea44,0x4bdecfa9,0xf6bb4b60,0xbebfbc70,
      0x289b7ec6,0xeaa127fa,0xd4ef3085,0x04881d05,
      0xd9d4d039,0xe6db99e5,0x1fa27cf8,0xc4ac5665,
      0xf4292244,0x432aff97,0xab9423a7,0xfc93a039,
      0x655b59c3,0x8f0ccc92,0xffeff47d,0x85845dd1,
      0x6fa87e4f,0xfe2ce6e0,0xa3014314,0x4e0811a1,
      0xf7537e82,0xbd3af235,0x2ad7d2bb,0xeb86d391
   };

   for(int i=0;i<16;i++)
      M[i] = (uint32_t)data[i*4] | ((uint32_t)data[i*4+1] << 8) |
           ((uint32_t)data[i*4+2] << 16) | ((uint32_t)data[i*4+3] << 24);

   a = ctx->h[0];
   b = ctx->h[1];
   c = ctx->h[2];
   d = ctx->h[3];

   for(int i=0;i<64;i++) {
      if(i < 16) { f = (b & c) | ((~b) & d); g = i; }
      else if(i < 32) { f = (d & b) | ((~d) & c); g = (5*i + 1) % 16; }
      else if(i < 48) { f = b ^ c ^ d; g = (3*i + 5) % 16; }
      else { f = c ^ (b | (~d)); g = (7*i) % 16; }
```

```
        temp = d;
        d = c;
        c = b;
        b = b + leftrotate(a + f + K[i] + M[g], s[i]);
        a = temp;
    }

    ctx->h[0] += a;
    ctx->h[1] += b;
    ctx->h[2] += c;
    ctx->h[3] += d;
}

void md5_init(MD5_CTX *ctx) {
    ctx->len = 0;
    ctx->h[0] = 0x67452301;
    ctx->h[1] = 0xefcdab89;
    ctx->h[2] = 0x98badcfe;
    ctx->h[3] = 0x10325476;
}

void md5_update(MD5_CTX *ctx, uint8_t *data, size_t len) {
    size_t i;
    for(i=0;i<len;i++) {
        ctx->buf[ctx->len % 64] = data[i];
        ctx->len++;
        if((ctx->len % 64) == 0)
            md5_transform(ctx, ctx->buf);
    }
}

void md5_final(MD5_CTX *ctx, uint8_t *hash) {
    uint64_t bitlen = ctx->len * 8;
    size_t index = ctx->len % 64;

    ctx->buf[index++] = 0x80;
    if(index > 56) {
        while(index < 64) ctx->buf[index++] = 0;
        md5_transform(ctx, ctx->buf);
        index = 0;
    }

    while(index < 56) ctx->buf[index++] = 0;

    for(int i=0;i<8;i++)
        ctx->buf[56+i] = bitlen >> (8*i);

    md5_transform(ctx, ctx->buf);
```

```
    for(int i=0;i<4;i++) {
        hash[i*4] = ctx->h[i] & 0xff;
        hash[i*4+1] = (ctx->h[i] >> 8) & 0xff;
        hash[i*4+2] = (ctx->h[i] >> 16) & 0xff;
        hash[i*4+3] = (ctx->h[i] >> 24) & 0xff;
    }
}

void md5_mac(const char *key, const char *message, uint8_t *output) {
    MD5_CTX ctx;
    md5_init(&ctx);
    md5_update(&ctx, (uint8_t*)key, strlen(key));
    md5_update(&ctx, (uint8_t*)message, strlen(message));
    md5_final(&ctx, output);
}

int main() {
    char key[100], message[500];
    uint8_t mac[16];

    printf("Enter key: ");
    scanf("%s", key);
    printf("Enter message: ");
    scanf("%s", message);

    md5_mac(key, message, mac);

    printf("MD5: ");
    for(int i=0;i<16;i++)
        printf("%02x", mac[i]);
    printf("\n");

    return 0;
}
```

**OUTPUT**



1. **(b) Find a message authentication code for given variable size message by using SHA512 or SHA-256 hash Algorithm and also measure the time consumption for varying message size.**

### (b) MAC using SHA-256 / SHA-512 with Time Measurement

**Aim** To generate a MAC for variable-size messages using SHA-256 or SHA-512 and measure time consumption.

**Algorithm**

1. Start the program.
2. Read the message from the user.
3. Record start time using clock().
4. Apply SHA-256 (or SHA-512) hashing.
5. Record end time.
6. Compute time taken.
7. Display the hash and execution time.
8. Stop.

**CODE**

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>
#define ROTRIGHT(a,b) (((a) >> (b)) | ((a) << (32-(b))))
#define CH(x,y,z) (((x) & (y)) ^ (~(x) & (z)))
#define MAJ(x,y,z) (((x) & (y)) ^ ((x) & (z)) ^ ((y) & (z)))
#define EP0(x) (ROTRIGHT(x,2) ^ ROTRIGHT(x,13) ^ ROTRIGHT(x,22))
#define EP1(x) (ROTRIGHT(x,6) ^ ROTRIGHT(x,11) ^ ROTRIGHT(x,25))
#define SIG0(x) (ROTRIGHT(x,7) ^ ROTRIGHT(x,18) ^ ((x) >> 3))
#define SIG1(x) (ROTRIGHT(x,17) ^ ROTRIGHT(x,19) ^ ((x) >> 10))
typedef struct {
uint8_t data[64];
uint32_t datalen;
uint64_t bitlen;
uint32_t state[8];
} SHA256_CTX;
uint32_t k[64] = {
  0x428a2f98,0x71374491,0xb5c0fbcf,0xe9b5dba5,
  0x3956c25b,0x59f111f1,0x923f82a4,0xab1c5ed5,
  0xd807aa98,0x12835b01,0x243185be,0x550c7dc3,
```

```
  0x72be5d74,0x80deb1fe,0x9bdc06a7,0xc19bf174,
  0xe49b69c1,0xefbe4786,0x0fc19dc6,0x240ca1cc,
  0x2de92c6f,0x4a7484aa,0x5cb0a9dc,0x76f988da,
  0x983e5152,0xa831c66d,0xb00327c8,0xbf597fc7,
  0xc6e00bf3,0xd5a79147,0x06ca6351,0x14292967,
  0x27b70a85,0x2e1b2138,0x4d2c6dfc,0x53380d13,
  0x650a7354,0x766a0abb,0x81c2c92e,0x92722c85,
  0xa2bfe8a1,0xa81a664b,0xc24b8b70,0xc76c51a3,
  0xd192e819,0xd6990624,0xf40e3585,0x106aa070,
  0x19a4c116,0x1e376c08,0x2748774c,0x34b0bcb5,
  0x391c0cb3,0x4ed8aa4a,0x5b9cca4f,0x682e6ff3,
  0x748f82ee,0x78a5636f,0x84c87814,0x8cc70208,
  0x90befffa,0xa4506ceb,0xbef9a3f7,0xc67178f2
};

void sha256_transform(SHA256_CTX *ctx, const uint8_t data[]) {
   uint32_t a,b,c,d,e,f,g,h,i,t1,t2,m[64];

   for(i=0;i<16;i++)
      m[i]=(data[i*4]<<24)|(data[i*4+1]<<16)|(data[i*4+2]<<8)|(data[i*4+3]);
   for(;i<64;i++)
      m[i]=SIG1(m[i-2])+m[i-7]+SIG0(m[i-15])+m[i-16];

   a=ctx->state[0]; b=ctx->state[1]; c=ctx->state[2]; d=ctx->state[3];
   e=ctx->state[4]; f=ctx->state[5]; g=ctx->state[6]; h=ctx->state[7];

   for(i=0;i<64;i++){
      t1=h+EP1(e)+CH(e,f,g)+k[i]+m[i];
      t2=EP0(a)+MAJ(a,b,c);
      h=g; g=f; f=e; e=d+t1;
      d=c; c=b; b=a; a=t1+t2;
   }

   ctx->state[0]+=a; ctx->state[1]+=b; ctx->state[2]+=c; ctx->state[3]+=d;
   ctx->state[4]+=e; ctx->state[5]+=f; ctx->state[6]+=g; ctx->state[7]+=h;
}

void sha256_init(SHA256_CTX *ctx){
   ctx->datalen=0; ctx->bitlen=0;
   ctx->state[0]=0x6a09e667; ctx->state[1]=0xbb67ae85;
   ctx->state[2]=0x3c6ef372; ctx->state[3]=0xa54ff53a;
   ctx->state[4]=0x510e527f; ctx->state[5]=0x9b05688c;
   ctx->state[6]=0x1f83d9ab; ctx->state[7]=0x5be0cd19;
}

void sha256_update(SHA256_CTX *ctx,const uint8_t data[],size_t len){
   for(size_t i=0;i<len;i++){
```

```
        ctx->data[ctx->datalen]=data[i];
        ctx->datalen++;
        if(ctx->datalen==64){
            sha256_transform(ctx,ctx->data);
            ctx->bitlen+=512;
            ctx->datalen=0;
        }
    }
}

void sha256_final(SHA256_CTX *ctx,uint8_t hash[]){
    uint32_t i=ctx->datalen;

    if(ctx->datalen<56){
        ctx->data[i++]=0x80;
        while(i<56) ctx->data[i++]=0x00;
    } else{
        ctx->data[i++]=0x80;
        while(i<64) ctx->data[i++]=0x00;
        sha256_transform(ctx,ctx->data);
        memset(ctx->data,0,56);
    }

    ctx->bitlen+=ctx->datalen*8;
    ctx->data[63]=ctx->bitlen;
    ctx->data[62]=ctx->bitlen>>8;
    ctx->data[61]=ctx->bitlen>>16;
    ctx->data[60]=ctx->bitlen>>24;
    ctx->data[59]=ctx->bitlen>>32;
    ctx->data[58]=ctx->bitlen>>40;
    ctx->data[57]=ctx->bitlen>>48;
    ctx->data[56]=ctx->bitlen>>56;
    sha256_transform(ctx,ctx->data);

    for(i=0;i<4;i++){
        hash[i]=(ctx->state[0]>>(24-i*8))&0xff;
        hash[i+4]=(ctx->state[1]>>(24-i*8))&0xff;
        hash[i+8]=(ctx->state[2]>>(24-i*8))&0xff;
        hash[i+12]=(ctx->state[3]>>(24-i*8))&0xff;
        hash[i+16]=(ctx->state[4]>>(24-i*8))&0xff;
        hash[i+20]=(ctx->state[5]>>(24-i*8))&0xff;
        hash[i+24]=(ctx->state[6]>>(24-i*8))&0xff;
        hash[i+28]=(ctx->state[7]>>(24-i*8))&0xff;
    }
}

int main(){
```

```
    char key[]="secret";
    size_t sizes[]={64,256,1024,4096,16384};
    int n=5;

    for(int i=0;i<n;i++){
        size_t size=sizes[i];
        char *message=malloc(size);
        memset(message,'A',size);

        SHA256_CTX ctx;
        uint8_t hash[32];

        clock_t start=clock();

        sha256_init(&ctx);
        sha256_update(&ctx,(uint8_t*)key,strlen(key));
        sha256_update(&ctx,(uint8_t*)message,size);
        sha256_final(&ctx,hash);

        clock_t end=clock();
        double time_taken=(double)(end-start)/CLOCKS_PER_SEC;

        printf("Message Size: %zu bytes\n",size);
        printf("MAC: ");
        for(int j=0;j<32;j++)
            printf("%02x",hash[j]);
        printf("\nTime: %f seconds\n\n",time_taken);

        free(message);
    }
    return 0;
}
```

**OUTPUT**

```
Message Size: 64 bytes
MAC: 31eef5210bb02bd0b66ce56be0b8eecd451539f86e366dd72e66f1cc796e38d9
Time: 0.000004 seconds

Message Size: 256 bytes
MAC: 4a1b311132ac7e12020137fba1e7d08a6b63f7ffbe961ad29a75b8212b7ada8c
Time: 0.000007 seconds

Message Size: 1024 bytes
MAC: bf8fa29998c1b189788411fa153d8aa188c3cbb6ac501d73d0ef3c9b969bc48c
Time: 0.000021 seconds

Message Size: 4096 bytes
MAC: 390aa3e58aadc02aec509adb199f5e1f219fe66a3dbd404640f9ff49bfe8e5f8
Time: 0.000097 seconds

Message Size: 16384 bytes
MAC: 9b2b103251936cc0bf177f438c71382d8e3569f72d16ed44de7743177be81bcb
Time: 0.000335 seconds



...Program finished with exit code 0
Press ENTER to exit console.
```

2. **(a) Implement Diffie Hellman hey exchange protocol.**

**(a) Diffie–Hellman Key Exchange**

**Aim :** To implement the Diffie–Hellman key exchange protocol to securely generate a shared secret between two parties.

**Algorithm**

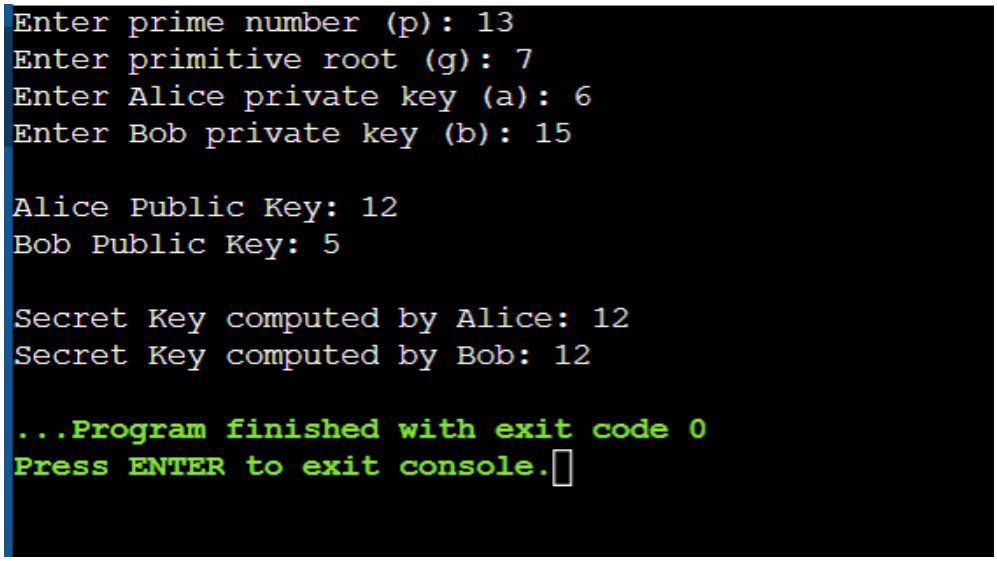
1. Start the program.
2. Choose a large prime number p and primitive root g.
3. User A selects private key a.
4. User B selects private key b.
5. Compute public keys: o A computes: $A=ga \bmod p$ o B computes: $B=gb \bmod p$
6. Exchange public keys.
7. Compute shared secret: o A computes: $K=Ba \bmod p$ o B computes: $K=Ab \bmod p$
8. Verify both secrets are equal.
9. Stop.

**CODE**

```
#include <stdio.h>
#include <math.h>

long long power(long long base, long long exp, long long mod) {
    long long result = 1;
    base = base % mod;
    while (exp > 0) {
        if (exp % 2 == 1)
```

```
        result = (result * base) % mod;
      exp = exp >> 1;
      base = (base * base) % mod;
   }
   return result;
}

int main() {
   long long p, g;
   long long a, b;
   long long A, B;
   long long keyA, keyB;

   printf("Enter prime number (p): ");
   scanf("%lld", &p);

   printf("Enter primitive root (g): ");
   scanf("%lld", &g);

   printf("Enter Alice private key (a): ");
   scanf("%lld", &a);

   printf("Enter Bob private key (b): ");
   scanf("%lld", &b);
A = power(g, a, p);
B = power(g, b, p);
printf("\nAlice Public Key: %lld", A);
printf("\nBob Public Key: %lld", B);
keyA = power(B, a, p);
keyB = power(A, b, p);
printf("\n\nSecret Key computed by Alice: %lld", keyA);
printf("\nSecret Key computed by Bob: %lld", keyB);
return 0;
}
```

**OUTPUT**



**(b) Implement a Digital Signature standard for verifying the legal communication parties**

**2(b) Digital Signature Standard (RSA-based)**

**Aim :**To implement a digital signature scheme to verify the authenticity and integrity of communicating parties.

**Algorithm**

**Key Generation**

1. Choose two prime numbers p and q.
2. Compute $n$=$p$×$q$.
3. Compute $\mathrm{phi}(n)$=($p$−1)($q$−1).
4. Choose public key e such that gcd(e, phi)=1.
5. Compute private key d such that

$\mathrm{d} * e \mathrm{mod}\ \mathrm{phi}(n)$=1.

**Signing**

6. Sender computes signature:

S=$M$^$d$ mod $n$.

**Verification**

7. Receiver computes:

M'=($S$^e )*mod $n$.

8. If $M'$=$M$, signature is valid.

**CODE**

```
#include <stdio.h>
#include <string.h>

long long power(long long base, long long exp, long long mod) {
   long long result = 1;
   base = base % mod;
   while (exp > 0) {
      if (exp % 2 == 1)
         result = (result * base) % mod;
      exp = exp >> 1;
      base = (base * base) % mod;
   }
   return result;
}

long long gcd(long long a, long long b) {
   while (b != 0) {
```

```
        long long temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

long long mod_inverse(long long e, long long phi) {
    long long t = 0, newt = 1;
    long long r = phi, newr = e;
    while (newr != 0) {
        long long q = r / newr;
        long long temp = newt;
        newt = t - q * newt;
        t = temp;
        temp = newr;
        newr = r - q * newr;
        r = temp;
    }
    if (t < 0)
        t += phi;
    return t;
}

long long simple_hash(char *msg) {
    long long hash = 0;
    for (int i = 0; msg[i] != '\0'; i++)
        hash += msg[i];
    return hash;
}

int main() {
    long long p = 11, q = 13;
    long long n = p * q;
    long long phi = (p - 1) * (q - 1);

    long long e = 7;
    while (gcd(e, phi) != 1)
        e++;

    long long d = mod_inverse(e, phi);

    char message[100];
    printf("Enter message: ");
    scanf("%s", message);
```

```
    long long hash = simple_hash(message) % n;
    printf("Hash value: %lld\n", hash);

    long long signature = power(hash, d, n);
    printf("Digital Signature: %lld\n", signature);

    long long verify = power(signature, e, n);
    printf("Decrypted Hash: %lld\n", verify);

if (verify == hash)
printf("Signature Verified. Message is Authentic.\n");
else
printf("Signature Verification Failed.\n");
return 0;
}
```

**OUTPUT**